콩잎 질병 분류 Vision AI 서비스

ResNet18 Head-only Fine-tuning + FastAPI 동기/비동기

콩잎 이미지 3클래스(angular_leaf_spot · bean_rust · healthy) 분류 모델을 학습하고, FastAPI 동기 API와 Celery 비동기 작업 큐, Redis 캐시, PostgreSQL 작업 기록으로 서비스화.

0.8346
Best **Validation** Accuracy (111/133, epoch 6)

0.8750
Test Accuracy (112/128, 1회 평가) · Macro F1 0.8752

21.8 ms / 1.5 ms
동기 응답 중앙값: 캐시 MISS / HIT

8.5 ms
비동기 접수 응답 중앙값 (202 + job_id)

항목	내용
최종 모델	`artifacts/beans_resnet18_bs16_ep6` (Head-only, batch 16, epoch 6)
비교 기준선	`artifacts/beans_resnet18` (batch 8, epoch 3, Val 0.7669, Test 0.7188)
실행 환경	Windows 11 + WSL2 Ubuntu 26.04, Python 3.14.4 (.venv), Docker: redis 7.4 · postgres 17
근거 문서	`reports/01~08_*.md`, `README.md`, `reports/data` · `evidence` · `figures`

발표자 Jaden · 실행·측정 2026-11-04

과제 개요와 데이터 구성

과제 범위

- 이미지 분류: 한 장을 3클래스 중 하나로 분류 (Bounding Box 탐지 아님)
- 사전학습 모델 `microsoft/resnet-18` → 출력층 1000 → 3클래스 교체 후 Fine-tuning
- 서빙: FastAPI 동기 API + Celery 비동기 큐, Redis 캐시·브로커, PostgreSQL 작업 이력
- 제공 코드는 수정하지 않고 실행 옵션만 변경. Test 평가 스크립트(`training/evaluate_test.py`)만 별도 추가

데이터 구성 (dataset/, .jpg 기준)

클래스	Train	Validation	Test	합계
angular_leaf_spot	345	44	43	432
bean_rust	348	45	43	436
healthy	341	44	42	427
합계	**1,034**	**133**	**128**	**1,295**

▶ 클래스별 개수가 거의 같은 균형 데이터. Train·Val 합계는 training_report.json의 1034 / 133과 일치.

분할 역할

분할	용도	사용 시점
Train	분류층 가중치 학습	매 epoch
Validation	저장 시점 선택, 하이퍼파라미터 비교	매 epoch 종료 후
Test	확정 모델 최종 평가 (학습 코드는 읽지 않음)	모델 확정 후 1회

▶ 학습 중 어떤 모델도 test 이미지를 읽지 않음. 모든 실험은 같은 분할을 그대로 사용.

데이터 누출 점검 (읽기 전용 실행)

점검	방법	결과
내용 중복	.jpg 1,295개 MD5 해시 비교, 분할 사이 같은 해시 탐색	**0건**
이름 중복	train-val, train-test, val-test 파일명 교집합	**0건**

▶ 한계: 바이트가 같은 파일만 탐지. 크기·압축만 다른 같은 사진은 탐지 불가.

> **발견 사항** · `train/healthy/healthy_train.120tore` 는 이미지가 아닌 macOS 폴더 메타데이터 파일. 학습 코드가 이미지 확장자만 읽어 자동 제외 → healthy Train은 파일 342개 중 이미지 **341장**.

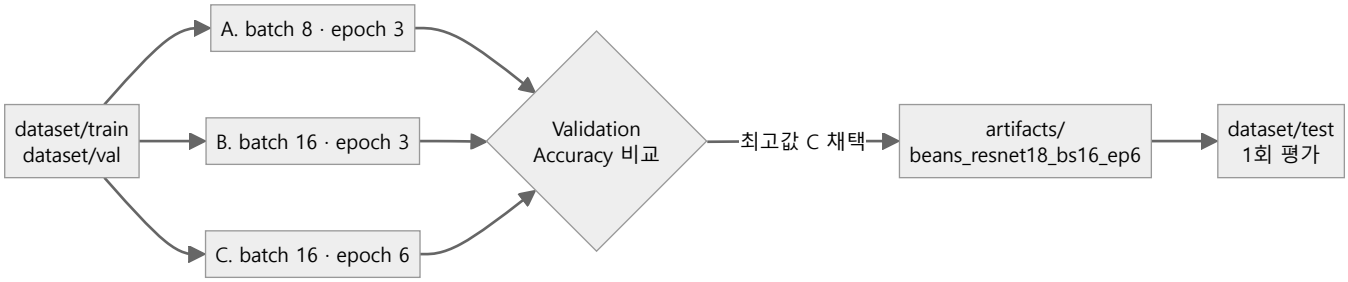
02 · 모델 학습

Head-only Fine-tuning과 모델 선택

Head-only 개념

- Backbone(합성곱 층)은 고정, 마지막 분류층 `classifier.*` 만 학습 (`--freeze-backbone`)
- 학습 파라미터: 512×3 + 3 = **1,539개** · 공통 설정: AdamW, lr **0.0002**, seed 42
- 저장 조건: Val Accuracy가 이전 최고보다 클 때만 저장 (`acc > best_acc`)

학습·선택 절차



도식 설명

- 세 설정 모두 `training/finetune.py` 를 옵션만 바꿔 실행, 매번 원본 resnet-18에서 시작
- 각 설정은 별도 output 폴더에 저장 → 기준선 모델을 덮어쓰지 않음
- 선택 기준은 Validation Accuracy만 사용. Test는 선택 뒤 `training/evaluate_test.py` 로 1회

하이퍼파라미터 비교 (Validation 기준)

실험	batch	epoch	Best Val Acc	결정
A. 기준선	8	3	**0.7669**	미채택
B. batch 변경	16	3	**0.7669**	미채택 (개선 없음)
C. batch 16 + epoch 증가	**16**	**6**	**0.8346**	**최종 채택**
참고: Full Fine-tuning	8	3	0.9549	터미널 로그 참고값

▶ batch 16은 epoch당 업데이트 횟수가 절반(약 130 → 65회) → 3 epoch로는 부족, 6 epoch에서 Val 0.8346까지 상승.

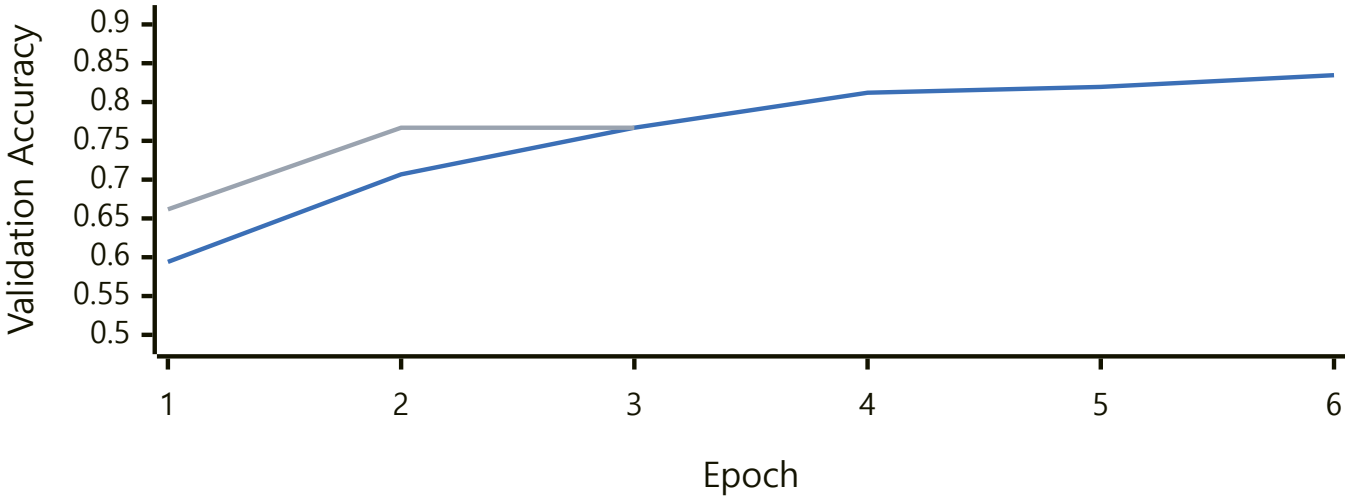
최종 모델 C: epoch별 Train Loss · Validation Accuracy

Epoch	1	2	3	4	5	6
Train Loss	1.0146	0.8475	0.7315	0.6579	0.6223	0.5717
Val Accuracy	0.5940	0.7068	0.7669	0.8120	0.8195	**0.8346**

▶ Loss는 매 epoch 감소, Val은 매 epoch 상승 → 6 epoch 모두 저장 조건 충족, 최종 파일은 epoch 6 모델.

Validation 곡선과 Test 결과

Validation Accuracy 곡선 (A vs C)



위쪽까지 이어진 선: 최종 C (batch 16, epoch 6) · 3 epoch에서 끝나는 선: 기준선 A (batch 8, epoch 3)

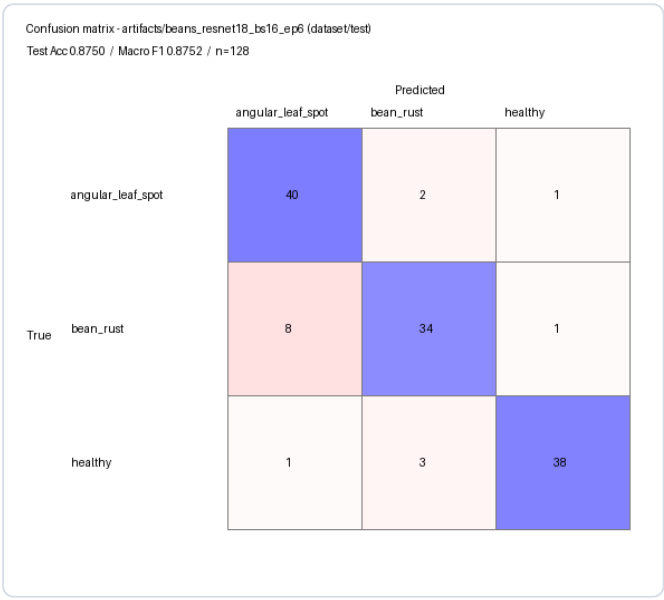
▶ A는 epoch 2 이후 0.7669에서 정체, C는 epoch 6까지 상승 중 → 최적 epoch는 미확인.

최종 모델 Test 결과 (2026-11-04 18:17, 1회)

클래스	n	P	R	F1
angular_leaf_spot	43	0.8163	0.9302	0.8696
bean_rust	43	0.8718	0.7907	0.8293
healthy	42	0.9500	0.9048	0.9268

Accuracy 0.8750 (112/128) · Macro F1 **0.8752**

▶ 가장 많은 오분류: bean_rust → angular_leaf_spot 8건. 기준선 A(Test 0.7188, bean_rust Recall 0.5349) 대비 개선.



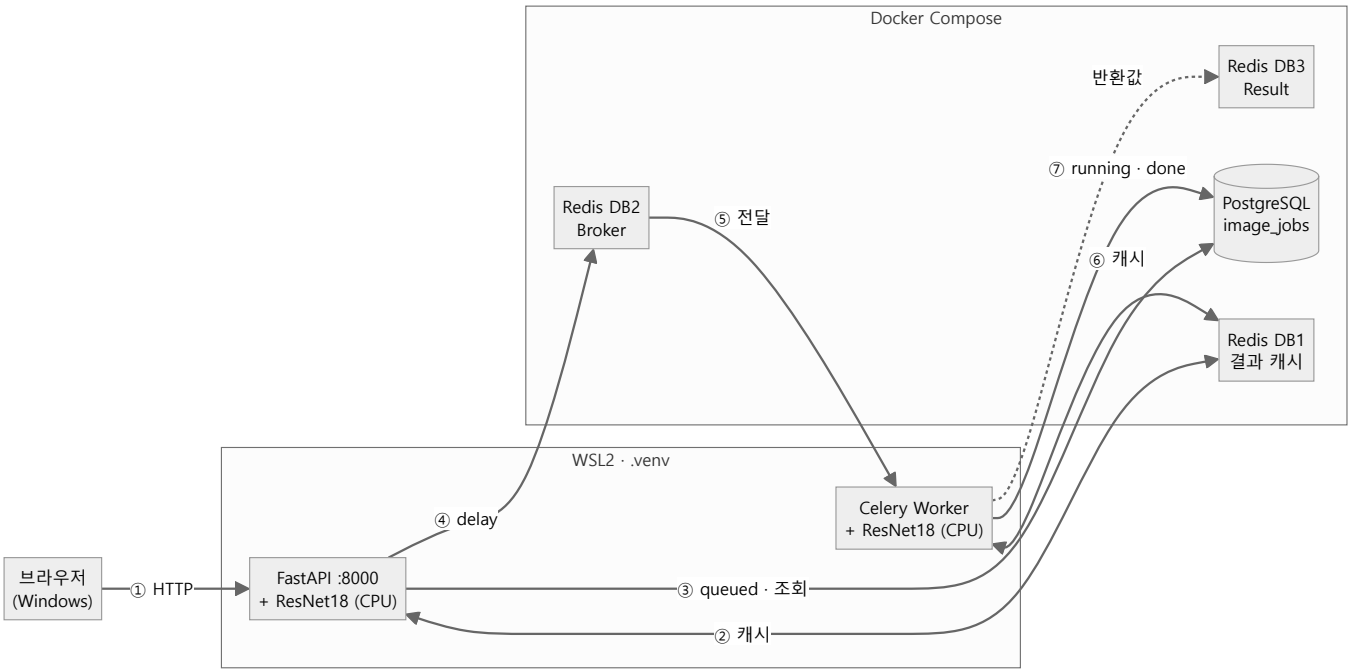
혼동행렬 (행: 정답, 열: 예측)

Test 사용 이력 (공개)

시각	모델	Test Acc	Macro F1
16:42	A. batch 8 · epoch 3	0.7188	0.7109
18:17	**C. batch 16 · epoch 6 (최종)**	**0.8750**	**0.8752**

▶ 같은 Test를 2회 사용. C는 Validation으로 선택했으나 A의 Test를 본 뒤 추가 실험 → C의 Test 점수에 낙관 편향 가능성.

전체 시스템 아키텍처



도식 설명

- ①②: 동기 요청은 FastAPI가 캐시 확인 후 MISS면 직접 추론 · ③~⑦: 비동기 요청은 DB 행 생성 → Broker → Worker가 처리·기록
- **FastAPI** (`main.py` , `ai/router_*.py`)와 **Celery Worker** (`ai/tasks.py`)는 Docker가 아닌 WSL .venv에서 실행. 비동기 업로드 이미지는 `media/jobs/{uuid}.bin` 에 저장 후 Worker가 읽음
- **ResNet18**은 두 프로세스가 각각 로드 (`ai/hf_model.py` , `lru_cache` , CPU) → 첫 요청 FastAPI **2,125 ms**, Worker **2,256 ms**
- **Redis**는 컨테이너 1개를 DB 번호로 분리: DB1 캐시(`ai/cache_service.py`), DB2 Broker, DB3 Result Backend(`ai/celery_app.py`). 상태 조회 API는 DB3이 아닌 PostgreSQL을 읽음
- **PostgreSQL** `image_jobs` 는 비동기 작업만 기록 (`ai/models.py`)
- compose.yaml의 web·worker·trainer 서비스는 이번 실행에서 사용하지 않음 (`docker compose up -d redis postgres`)

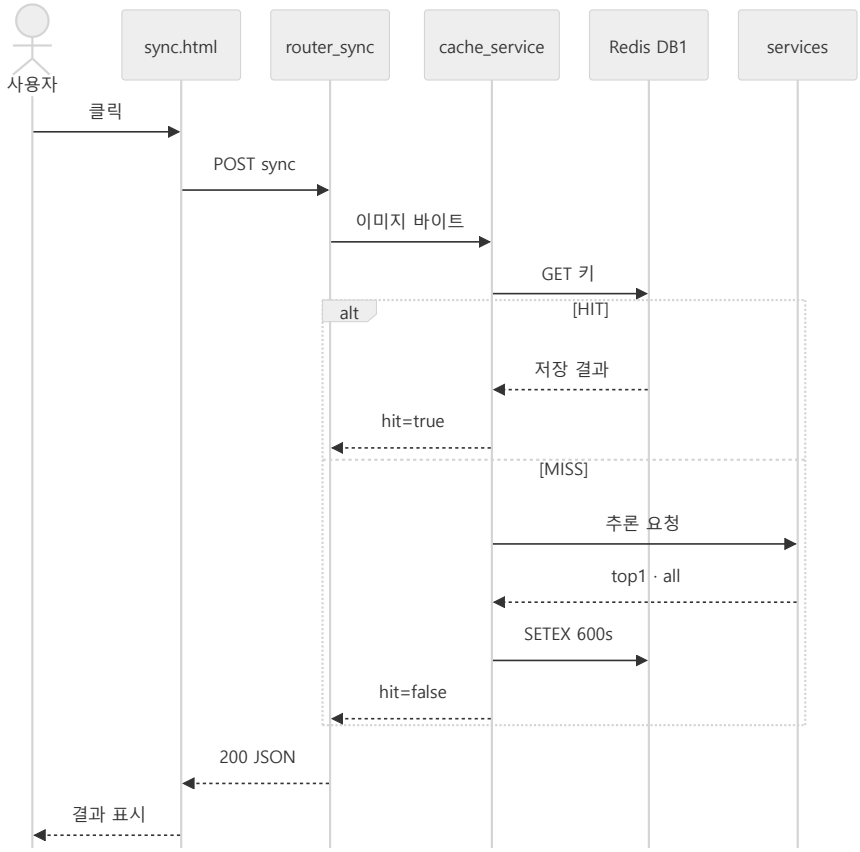
실행 환경

항목	값
OS	Windows 11 Pro (10.0.26200) + WSL2 Ubuntu 26.04.1 LTS
Python · 주요 패키지	3.14.4 · fastapi 0.162.0 · celery 5.6.3 · transformers 4.57.6 · torch 2.15.0
Docker	redis:7.4-alpine (127.0.0.1:6379) · postgres:17-alpine (127.0.0.1:5432)
하드웨어	i7-12700 · RAM 15GiB (WSL) · RTX 3060 (학습에 사용, 서비스 추론은 CPU)
모델 경로	`.env` MODEL_PATH=artifacts/beans_resnet18_bs16_ep6

▶ 모델 교체는 .env 변경 + Worker·Uvicorn 재시작만으로 반영 (응답 model_id로 확인).

04 · 처리 흐름

동기 처리 흐름: [동기 분류] 버튼

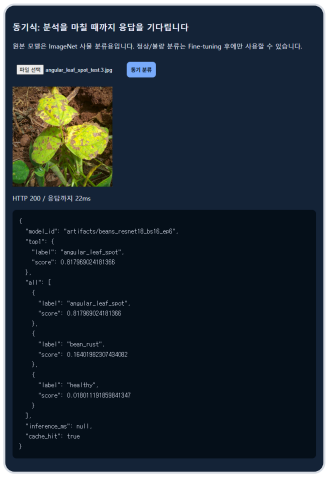


도식 설명

- `POST /api/sync/classify/` → `classify_sync()` : 5MB 초과면 413 · 브라우저(`sync.html`)는 응답까지 버튼 비활성화
- 캐시 키 = 모델 경로 해시 16자 + 이미지 SHA-256 → 모델을 바꾸면 같은 이미지도 MISS
- MISS일 때만 `services.classify_image_bytes()` : `load_rgb_image()` 검증(실패 시 400) → `get_classifier()` ResNet18 추론 (CPU) → 600초 캐시 저장
- HIT 응답은 `cache_hit true` , `inference_ms null`
- DB에는 기록하지 않음 (`router_sync.py` 에 DB 쓰기 없음)

사례 (최종 모델)	시간
MISS · bean_rust_test.0 → bean_rust 0.7914	21.1 ms
HIT · 같은 이미지 재요청	1.6~2.0 ms
프로세스 첫 요청 (모델 로드)	2,156 ms

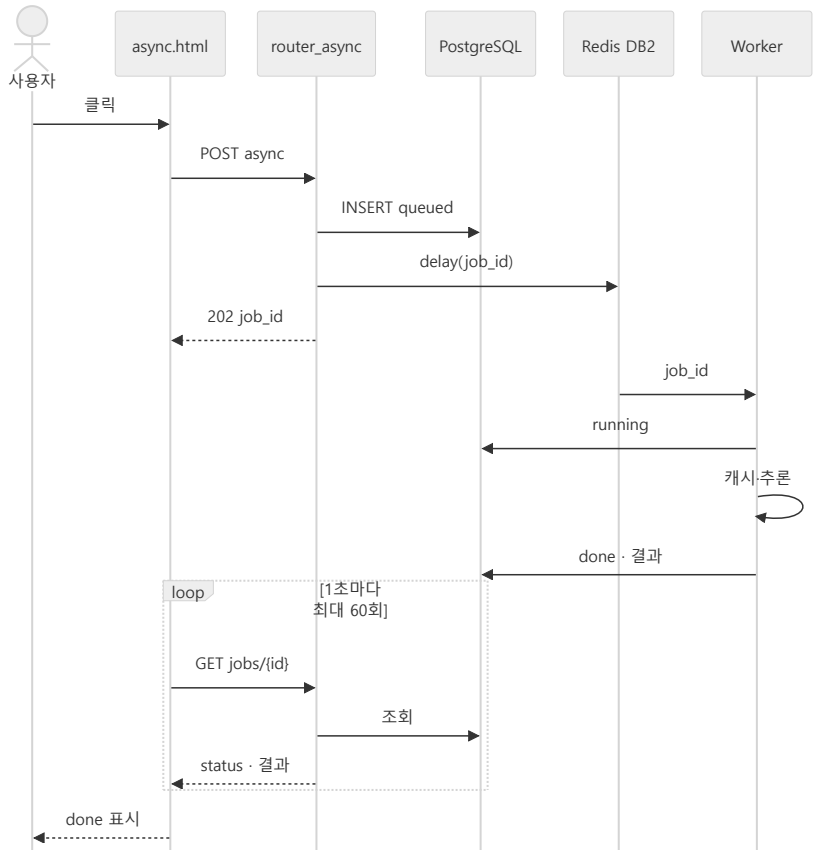
▶ 화면 예시는 600초 안의 재요청이라 cache_hit true, inference_ms null.



sync_page_bs16_ep6.png · angular_leaf_spot 0.8180

04 · 처리 흐름

비동기 처리 흐름: [Celery 작업 등록] 버튼



도식 설명

- `POST /api/async/classify/` → `enqueue_classify()` : 검증(400·413) → `media/jobs/{uuid}.bin` 저장 → queued 행 → `classify_job.delay()` → 202 + job_id · check_url
- Broker 등록 실패 시 행을 failed로 바꾸고 503 응답 (코드 확인, 실행은 미실시)
- Worker(`ai/tasks.py`)는 동기와 같은 `classify_with_cache()` 를 호출, 예외 시 failed + error_message
- `async.html` 은 1초 대기 후 `GET /api/async/jobs/{id}/` (`get_job()`) 반복 → 화면의 전체 시간은 최소 1초

사례 (최종 모델)	시간
job 16 접수 응답 (202)	29.5 ms
job 16 생성→done (Worker 첫 작업)	2.31 s
이후 MISS 작업 생성→done	약 29 ms
화면 job 27: 접수 / 전체	29 / 1,046 ms

▶ job 27(bean_rust_test.3)은 healthy 0.3727로 오분류 → 기능 정상, 모델 한계.



async_page_bs16_ep6.png · job 27 done

API 명세 요약

ID	Method · URL	요청	성공	주요 응답	실패
API-01	GET `/health`	없음	200	status	—
API-02	POST `/api/sync/classify/`	multipart image	200	model_id, top1, all, inference_ms, cache_hit	400 413 422
API-03	POST `/api/async/classify/`	multipart image	202	job_id, status, check_url	400 413 422 503
API-04	GET `/api/async/jobs/{id}/`	정수 job_id	200	status, top1, all, error_message	404 422
WEB	GET `/vision/sync/` · `/vision/async/` · `/`	없음	200 · 307	HTML · /vision/sync/로 이동	—

▶ 입력 조건: JPG·PNG·WEBP 실제 형식, 5MB 이하, 1200만 픽셀 이하. 모든 상태코드는 최종 모델로 실제 호출해 확인 (503 제외).

상태코드 의미

코드	의미	실제 확인 입력	응답 detail
200	처리 완료 (동기 결과, 상태 조회)	test 이미지	결과 JSON
202	**접수**만 완료, 분류는 Worker에서 진행	bean_rust_test.1	job_id, check_url
400	이미지가 아니거나 형식·해상도 위반	.txt 파일	올바른 이미지 파일이 아닙니다.
404	없는 작업 번호	job 999999	해당 작업이 없습니다.
413	5MB 초과	6,000,000 B 파일	5MB 이하 파일만 허용합니다.
422	필드 누락·경로 변수 형식 오류 (FastAPI 자동)	image 없음, job_id=abc	Field required 등
503	큐 등록 실패 (Redis Broker 연결 불가)	미실시 (모의 테스트로만 확인)	작업 큐 등록 실패

▶ Swagger(/docs)에는 200·202·422만 표시 → response_model·오류 응답 미선언. 명세서는 실제 응답으로 보완.

응답 예시 (최종 모델, 동기 MISS)

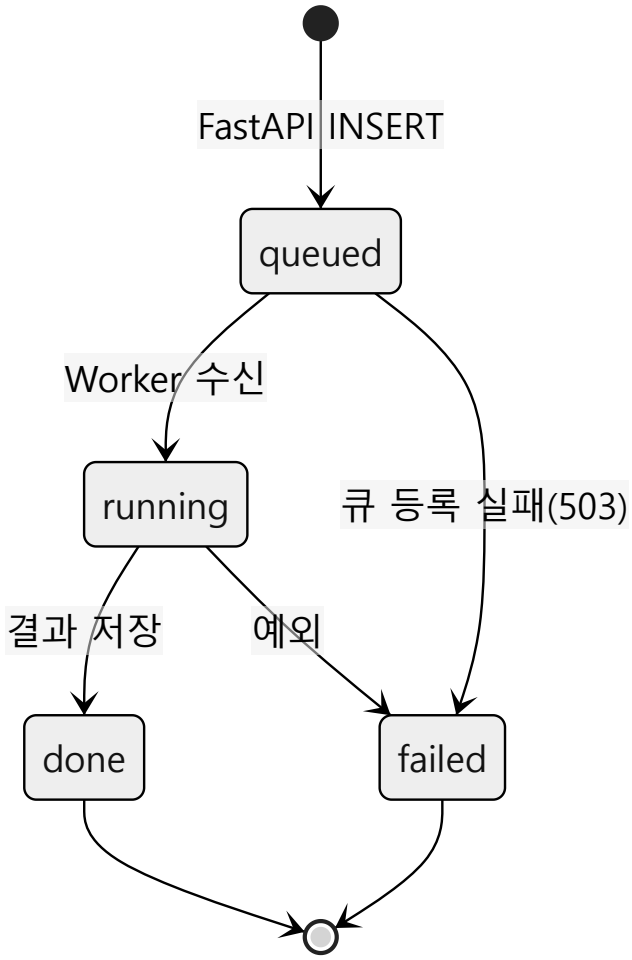

```
{"model_id":"artifacts/beans_resnet18_bs16_ep6","top1":{"label":"bean_rust","score":0.7914},
 "all":[bean_rust 0.7914, healthy 0.1250, angular_leaf_spot 0.0836],"inference_ms":16.79,"cache_hit":false}
```


▶ 캐시 HIT 응답은 같은 점수에 inference_ms null, cache_hit true (null은 0 ms가 아니라 추론을 실행하지 않았다는 뜻).

06 · DB

DB 구조: image_jobs와 상태 전이

IMAGE_JOBS		
int	id	PK
varchar	image_path	
varchar	status	
varchar	model_id	
varchar	top1_label	
double	top1_score	
json	all_scores	
double	inference_ms	
boolean	cache_hit	
text	error_message	
timestamptz	created_at	
timestamptz	updated_at	



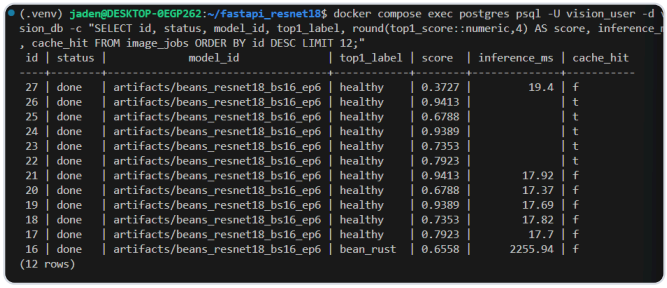
도식 설명

- 테이블 1개, PK `id`, `status` 인덱스. FK·관계 테이블 없음 (`ai/models.py`, `\d image_jobs` 로 확인)
- status·model_id·시각 기본값은 DB가 아니라 SQLAlchemy(Python)가 INSERT 때 채움
- queued는 `router_async.py`, running·done·failed는 `tasks.py` 가 기록
- 응답 필드 `top1` = top1_label + top1_score, `all` = all_scores · 시각은 UTC 저장

실제 기록 (최종 모델)

id	내용	생성→done
16	Worker 첫 작업, bean_rust 0.6558	2.31 s
17~21	MISS, healthy, 추론 17~18 ms	약 29 ms
22~26	HIT, inference_ms NULL	약 9 ms
27	브라우저 화면, healthy 0.3727	—

▶ 동기 16회 + 비동기 11회 → 새 행 11개 · 동기 요청은 행을 만들지 않음 · failed 행 미발생.



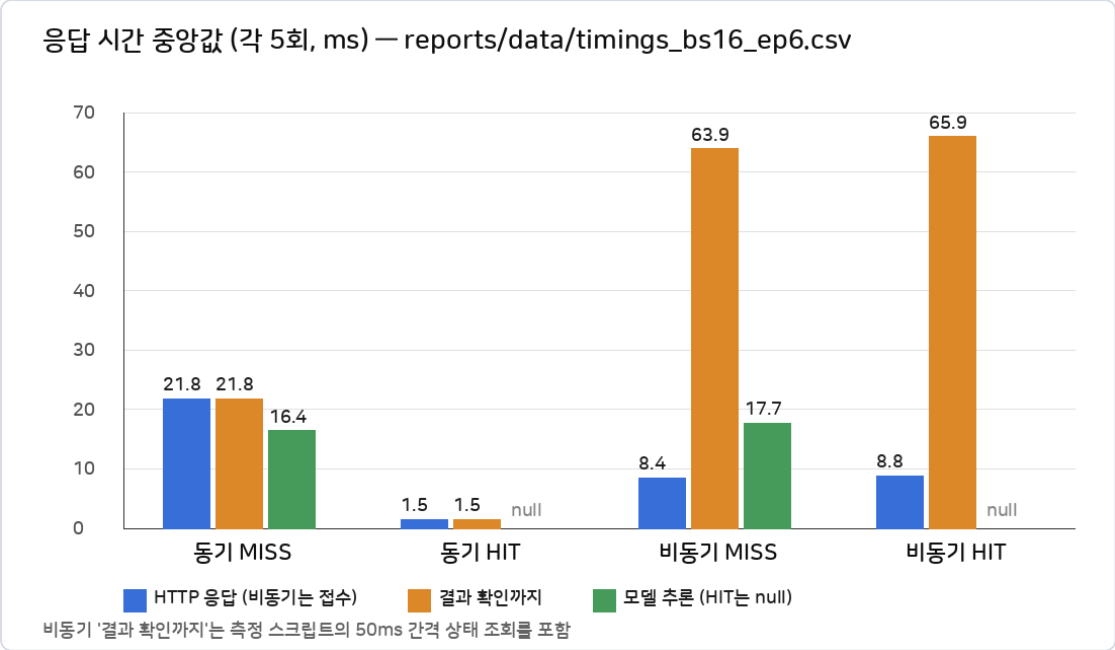
db_rows_bs16_ep6.png

07 · 성능

성능 측정과 비교 (최종 모델, 조건별 5회)

경로	캐시	HTTP 응답	결과 확인까지	추론	DB 기준 처리
동기	MISS	21.81 ms	21.81 ms	16.42 ms	—
동기	HIT	1.48 ms	1.48 ms	null	—
비동기	MISS	8.45 ms (접수)	63.91 ms	17.70 ms	29.13 ms
비동기	HIT	8.83 ms (접수)	65.94 ms	null	8.94 ms

▶ 중앙값 · 워밍업(첫 요청) 제외 · 같은 PC에서 httpx로 측정 · 원본 reports/data/timings_bs16_ep6.csv.



계산

항목	식	결과
HTTP 응답 감소율 (MISS 대 MISS)	(21.81 − 8.45) / 21.81	**61.26%**
결과 확인 시간 차이	63.91 − 21.81	**+42.10 ms**
동기 캐시 HIT 감소율	(21.81 − 1.48) / 21.81	**93.21%**
Worker 처리 캐시 효과 (DB 기준)	(29.13 − 8.94) / 29.13	**69.31%**

해석

- 비동기 접수가 빠른 것은 접수와 처리를 분리한 효과이며 **추론 가속이 아님** (추론 16.42 vs 17.70 ms)
- 캐시 HIT의 `inference_ms = null`은 0 ms가 아니라 추론 미실행 → 평균 계산에서 제외
- 비동기 결과 확인은 50 ms 조회 간격에 묶여 MISS·HIT 차이가 드러나지 않음 (DB 기준으로만 확인)
- 기준선 A와 응답 시간 차이 1~2 ms 이내 → 모델 교체는 속도가 아닌 정확도 개선

기능 테스트 결과와 오류 이력

실환경 통합 테스트 (최종 모델, curl)

ID	확인 항목	실제 결과	판정
T-01	FastAPI 헬스	200 ok	PASS
T-02	동기 3클래스 분류	3/3 정답, model_id bs16_ep6	PASS
T-03·04	비동기 접수·완료	202 → queued → running → done (job 16)	PASS
T-05·06	Redis·DB 연결, DB 기록	PONG, accepting, id 16~26 done	PASS
T-07·08	불량 입력, 없는 작업	400, 404	PASS
T-09	캐시 반복	MISS → HIT	PASS
T-11~14	413, 422 ×2, 화면 200·307	모두 기대값	PASS
T-10·15	Worker 정지, Redis 중지(503)	실행하지 않음	미실시

▶ 13개 PASS · 2개 미실시 (기준선 A 때와 같은 결과). 모의 테스트 pytest 4 passed는 가짜 모델·임시 SQLite라 별도 기록.

오류 이력

ID	현상	원인	조치 · 재검증
E-01	docker: command not found	Docker Desktop 미실행	Docker Desktop 실행 → compose ps 정상
E-02	no pq wrapper available	psycopg-binary 없음	psycopg[binary] 설치 → impl binary, 서버 시작
E-03	8000 address already in use	web 컨테이너가 포트 점유	web·worker 중지, redis·postgres만 실행
E-04	No module named 'ai'	sys.path에 루트 없음	루트에서 python -m pytest → 4 passed
E-05	옵션 없이 학습 실행	Full FT가 기본 폴더에 저장	Head-only 재실행, 실험별 output 분리
E-06	기준선 정확도 부족	3 epoch 학습 부족	bs16·ep6 채택 → 서비스 교체 후 13 PASS

▶ 오류 당시 화면은 남기지 않음 → 작업 기록과 셀 기록 기준으로 작성.

관찰된 오분류 (기능 오류 아님)

- angular_leaf_spot_test.4 → bean_rust 0.4729 (성능 측정 중)
- bean_rust_test.3 → healthy 0.3727 (job 27, 기준선 A에서도 healthy)

한계와 OpenCV 개선 제안

한계

구분	내용
Test 사용	같은 Test 2회 사용 → 최종 Test 점수 낙관 편향 가능
학습	epoch 6에서도 Val 상승 중 → 최적 epoch 미확인 · Full FT(Val 0.9549 참고값) 산출물 미보관
분류 품질	bean_rust Recall 0.7907, angular_leaf_spot과 혼동 8건
추적성	model_id는 경로만 기록 → 같은 경로의 파일 교체는 구분 불가
성능 측정	5회 반복, 같은 PC, 동시 부하 미측정, 50 ms 조회 단위
테스트	Worker 정지·Redis 중지(503)는 실환경 미실시

▶ 한계는 모두 보고서 01·06·07·08에 같은 내용으로 기록.

OpenCV 개선 제안 (설계만, 미구현 · 개선 실측 없음)

제안	내용	근거가 된 관찰
1. HSV 병반 면적비	HSV 변환 → 잎 마스크 → 갈색·황색 병반 후보 비율과 윤곽 수	질병↔healthy 혼동 (Test 6건)
2. review_required 규칙	top1 점수가 낮거나 1·2위 차가 작으면 사람 확인으로 분기	오분류 사례 점수 0.37~0.47
3. CNN + OpenCV ML 융합	3클래스 점수 + 병반 특징 → 보조 분류기	bean_rust → angular 8건

▶ 기준값·보조 모델은 Train/Val로만 정하고, 검증에는 새 평가 데이터 필요 (현재 Test는 이미 2회 사용).

다음 단계

- 하이퍼파라미터 후보를 먼저 정하고 Validation으로만 비교, Test는 최종 1회
- epoch 확대 실험으로 최적 epoch 확인 (Validation 기준)
- 모델 파일 SHA-256을 응답·DB에 함께 기록

10 · 시연 · Q&A

시연 순서와 예상 질문

시연 체크리스트 (서버 기동 후)

- ☐ `docker compose ps` → redis·postgres healthy, Worker·Uvicorn 실행 확인
- ☐ /vision/sync/ 에서 test 이미지 선택 → [동기 분류] → 200, model_id bs16_ep6, **cache_hit false**
- ☐ 같은 이미지로 한 번 더 → **cache_hit true**, inference_ms null (MISS → HIT)
- ☐ /vision/async/ 에서 다른 이미지 → [Celery 작업 등록] → **202 + job_id**
- ☐ 상태가 queued/running → **done**으로 바뀌는 것 확인 (Worker 터미널 received → succeeded)
- ☐ `psql ... ORDER BY id DESC LIMIT 5` → 방금 job_id 행 done 확인
- ☐ (선택) /docs Swagger, .txt 업로드로 400 확인

예상 질문

Q1. 왜 Full Fine-tuning이 아닌 Head-only인가?
과제 기준 명령이 Head-only. Full은 로그상 Val 0.9549였지만 산출물을 남기지 않아 Test 비교를 하지 못함.

Q2. batch 16·epoch 6은 어떻게 골랐나?
세 설정을 Validation Accuracy로만 비교해 최고값(0.8346) 채택. 학습 코드는 test를 읽지 않음.

Q3. Test 점수 0.8750을 그대로 믿어도 되나?
같은 Test를 기준선에도 1회 사용 → 낙관 편향 가능성을 보고서에 공개. 새 평가 데이터로 재확인 필요.

Q4. 비동기가 더 빠른가?
접수 응답(8.45 ms)만 빠름. 결과 확인까지는 42.10 ms 더 느림. 추론 시간은 동일.

Q5. 캐시 HIT의 inference_ms가 null인 이유는?
추론을 실행하지 않고 Redis 결과를 반환했기 때문. 0 ms라는 뜻이 아님.

Q6. 동기 결과는 DB에 남나?
남지 않음. image_jobs는 비동기 작업만 기록 (동기 16회 + 비동기 11회 → 행 11개).

Q7. Redis가 꺼지면?
동기는 캐시를 건너뛰고 추론. 비동기 등록은 failed 기록 후 503 (코드·모의 테스트로 확인).

Q8. 모델을 바꾸려면?
.env의 MODEL_PATH 변경 후 Worker·Uvicorn 재시작. 캐시 키에 모델 경로가 들어가 이전 결과는 재사용되지 않음.

최종 점검

검증 결과 요약

✅ 검증 통과

- 데이터 분할 사이 중복 0건 (MD5·파일명)
- Head-only Fine-tuning 3개 설정 비교 → Validation 기준 최종 모델 채택 (Val **0.8346**)
- 최종 모델 Test **0.8750** (112/128), Macro F1 **0.8752**
- 서비스 응답 model_id = `artifacts/beans_resnet18_bs16_ep6` 확인
- 동기·비동기 API, Redis 캐시, Celery, PostgreSQL 기록 실환경 **13개 PASS**
- 동기 결과 DB 미저장, 캐시 HIT 시 추론 생략, 상태 전이 queued → running → done 확인
- 모든 수치의 원본 파일 보관: training_report.json, test_metrics, timings CSV, raw 응답, DB 조회

⚠ 주의할 점

- Test를 기준선과 최종 모델에 2회 사용 → 최종 Test 점수 낙관 편향 가능
- 비동기 접수 61.26% 단축은 추론 가속이 아님 (추론 시간 동일)
- HIT의 inference_ms null ≠ 0 ms
- Full Fine-tuning Val 0.9549는 로그 참고값 (모델·Test 결과 없음)
- Worker 정지·Redis 중지(503) 시나리오는 실환경 미실시
- OpenCV 개선은 설계 제안만, 개선 수치 주장 없음

제출물 위치

구분	경로
보고서	reports/01_FineTuning_결과보고서 ~ 08_최종기술보고서.md · README.md
측정 원본	reports/data/ (test_metrics, timings_bs16_ep6.csv, timings.csv)
증빙	reports/evidence/ (캡처, raw_bs16_ep6/, raw/) · reports/figures/
모델	artifacts/beans_resnet18_bs16_ep6 (최종) · beans_resnet18 · beans_resnet18_bs16 (비교)